

ბურთულების მანქანა

მადინამ IOI-ის მონაწილეების გასართობად ბურთულების მანქანა გამოიგონა. მანქანის შიდა სტრუქტურა შემდეგნაირია:

- მანქანა შედგება N ცალი **კვანძისგან**, რომლებიც გადანომრილია 0 -დან $N - 1$ -მდე. $N - 1$ -ე კვანძს **სათავე**(root) ეწოდება.
- თითოეული u კვანძისთვის, რომლის მნიშვნელობებია $0 \leq u < N - 1$, u კვანძის **მშობელი** არის $P[u]$ კვანძი, რომლისთვისაც $P[u] > u$, ხოლო u კვანძი არის $P[u]$ -ის **შვილი**. სათავეს მშობელი არ ჰყავს. კვანძს, რომელსაც შვილები არ აქვს, **ფოთოლი** ეწოდება.

თქვენ **არ** იცით არც N და არც რომელიმე u კვანძის მშობელი $P[u]$. ამის ნაცვლად, მადინა გეუბნებათ M -ს, ფოთლების რაოდენობას. ასევე იცით, რომ ფოთლების ინდექსებია $0, 1, \dots, M - 1$. თქვენი ამოცანაა, განსაზღვროთ მანქანის შიდა სტრუქტურა თქვენს მიერ ჩატარებული ოპერაციების შედეგებზე დაკვირვებით.

მანქანის თითოეულ კვანძში შეიძლება განთავსდეს მაქსიმუმ ერთი **ბურთულა**, ხოლო თითოეულ ბურთულას აქვს **ღირებულება**, რომელიც მთელი არაუარყოფითი რიცხვია. ვამბობთ, რომ კვანძს აქვს ღირებულება x , თუ მასში განთავსებულია x ღირებულების ბურთი. კვანძი **დაკავებულია**, თუ ის ბურთულას შეიცავს; სხვა შემთხვევაში, ის **თავისუფალია**. თავდაპირველად, ყველა კვანძი თავისუფალია.

შეგიძლიათ შეასრულოთ ორი სახის ოპერაცია:

- **insert**(U, X): ახალ ბურთზე დაწერეთ მნიშვნელობა X და სცადეთ მისი ჩასმა ფოთოლ U -ში.
 - თუ ფოთოლი U დაკავებულია, ოპერაცია უბრუნებს `false` ბურთის ჩაყენებლად.
 - თუ ფოთლის კვანძი U თავისუფალია, ბურთი თავსდება კვანძში U . შემდეგ ბურთი განმეორებით გადაადგილდება მისი ახლანდელი კვანძის მშობლიურ კვანძში, სანამ ეს მშობლიური კვანძი თავისუფალია. გადაადგილება ჩერდება, როდესაც ბურთი მიაღწევს სათავეს ან კვანძს, რომლის მშობლიური კვანძი დაკავებულია. ოპერაცია აბრუნებს `true`.
- **collect**($\cdot$): თუ სათავე თავისუფალია (რაც ნიშნავს, რომ მანქანა ცარიელია), `collect` ცარიელ მასივს აბრუნებს. სხვა შემთხვევაში, ქვემოთ აღწერილი რეკურსიული პროცედურა `traverse` აგროვებს მანქანაში ამჟამად მყოფი ყველა ბურთის მნიშვნელობებს მასივში S . თავდაპირველად, მასივი S ცარიელია და გამოიძახება `traverse(N - 1)`.

```
traverse(u):
```

დაამატე u კვანძში არსებული ბურთულის ღირებულება s მასივის ბოლოში.

u კვანძის დაკავებული შვილების სია იყოს $c[u]$.

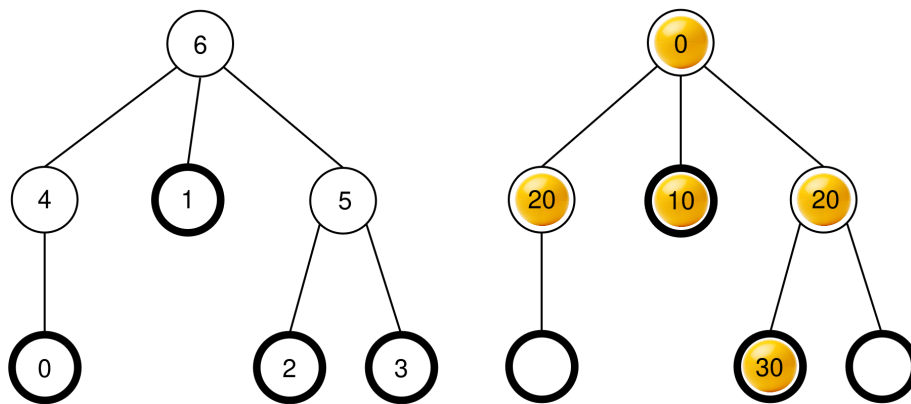
დავასორტიროთ $c[u]$ ღირებულებების არაკლებადი თანმიმდევრობით (თუ რამდენიმე კვანძს აღმოაჩნდება თანაბარი ღირებულების ბურთულები, ისინი შეიძლება დალაგდნენ ნებისმიერი თანმიმდევრობით).

```
for each v in c[u]:
```

```
    traverse(v)
```

ამ პროცედურის დასრულების შემდეგ, მანქანა დაიცლება ბურთებისგან, და `collect` დააბრუნებს S .

მაგალითად, განვიხილოთ მანქანა $N = 7$ კვანძით და მშობლების მასივით $P = [4, 6, 5, 5, 6, 6]$. მარცხენა სურათზე ნაჩვენებია ცარიელი მანქანა კვანძების ინდექსებით, ხოლო მარჯვენა სურათზე — ბურთების შესაძლო კონფიგურაცია `insert` ოპერაციების გარკვეული თანმიმდევრობის შემდეგ (ეს თანმიმდევრობა მოცემულია მაგალითების სექციაში).



როდესაც `collect()` გამოიძახება, სათავე(კვანძი 6) დაკავებულია, ამიტომ S ინიციალიზდება როგორც $S = []$ და იძახება `traverse(6)`.

`traverse(6)`: კვანძს 6 აქვს ღირებულება 0; მისი S -ს დამატება იძლევა $S = [0]$. კვანძი 6-ის შვილები არიან 4, 1, და 5, რომელთაგან ყველა დაკავებულია. მათი მნიშვნელობებია, შესაბამისად, 20, 10, და 20. არაკლებადი თანმიმდევრობით დალაგებისას მივიღებთ $c[6] = [1, 5, 4]$ (გაითვალისწინეთ, რომ $c[6] = [1, 4, 5]$ ასევე მისაღები იქნებოდა, რადგან კვანძებს 4 და 5 ერთნაირი ღირებულება აქვთ). ამის შემდეგ პროცედურა $c[6]$ -ში არსებულ თითოეულ კვანძზე ამ თანმიმდევრობით იძახებს `traverse`-ს.

- **`traverse(1)`:** კვანძს 1 აქვს მნიშვნელობა 10; მისი S -ის ბოლოში დამატებით მივიღებთ $S = [0, 10]$ -ს. კვანძს 1 არ ჰყავს დაკავებული შვილები, ამიტომ ეს გამოძახება სრულდება.
- **`traverse(5)`:** კვანძს 5 აქვს მნიშვნელობა 20; მისი S -ის ბოლოში დამატებით მივიღებთ $S = [0, 10, 20]$. კვანძს 5 აქვს ერთი დაკავებული შვილი, კვანძი 2, ამიტომ $c[5] = [2]$ და პროცედურა იძახებს `traverse(2)`.
 - **`traverse(2)`:** კვანძს 2 აქვს მნიშვნელობა 30; მისი S -ის ბოლოში დამატებით მივიღებთ $S = [0, 10, 20, 30]$. კვანძს 2 არ ჰყავს დაკავებული შვილები, ამიტომ ეს გამოძახება სრულდება.

- პროცედურა უბრუნდება `traverse(5)`-ს, რომელმაც დაამუშავა `c[5]`-ში არსებული ყველა შვილი, ამიტომ მისი გამოძახება სრულდება.
- **`traverse(4)`**: კვანძს 4 აქვს მნიშვნელობა 20; მისი S -ის ბოლოში დამატებით მიიღება $S = [0, 10, 20, 30, 20]$. კვანძს 4 არ ჰყავს დაკავებული შვილები, ამიტომ ეს გამოძახება სრულდება.

ყველა გამოძახება დასრულდა და დასამუშავებლად მეტი კვანძი არ დარჩა. საბოლოოდ, მანქანა დაიწალა ბურთულებისგან და `collect` აბრუნებს მასივს $S = [0, 10, 20, 30, 20]$.

თქვენი ამოცანაა, განსაზღვროთ კვანძების რაოდენობა N და წარმოადგინოთ მშობლების მასივი $R = [R[0], R[1], \dots, R[N - 2]]$, რომელიც აღწერს მანქანის სტრუქტურას, მაგრამ იმ კვანძების გადანომრვა, რომლებიც არ არიან ფოთლები და არც სათავე, დასაშვებია იყოს განსხვავებული. ფორმალურად, წარმოდგენილი მასივი R მიიჩნევა სწორად, თუ შესაძლებელია მანქანის თითოეულ კვანძს u მიენიჭოს უნიკალური იდენტიფიკატორი $L[u]$, სადაც $0 \leq L[u] < N$, ისე, რომ:

- $L[u] = u$ ყველა $0 \leq u < M$ და $u = N - 1$ -სთვის, და
- $L[P[u]] = R[L[u]]$ ყველა $0 \leq u < N - 1$ -სთვის.

აუცილებელი არ არის, რომ $R[u] > u$. მანქანა უნდა იყოს ცარიელი, როდესაც თქვენს პასუხს დააბრუნებთ.

დავუშვათ, K არის შესრულებული `collect` ოპერაციების რაოდენობა, ხოლო B — ყველა `insert` გამოძახების განმავლობაში ყველა ბურთულაზე დაწერილი მნიშვნელობებს შორის მაქსიმალური. ვთქვათ, $C = K + B$. მაშინ C არ უნდა აღემატებოდეს 1000-ს. თქვენი ქულა ზოგიერთ ქვეამოცანაში დამოკიდებულია C -ის მნიშვნელობაზე.

იმპლემენტაციის დეტალები

თქვენ იმპლემენტაცია უნდა გაუკეთოთ შემდეგ პროცედურას.

```
std::vector<int> find_structure(int M)
```

- M : მანქანაში ფოთლების რაოდენობა.
- პროცედურა გამოიძახება ზუსტად ერთხელ თითოეული ტესტისათვის.
- პროცედურამ უნდა დააბრუნოს $N - 1$ ზომის მასივი $R = [R[0], R[1], \dots, R[N - 2]]$, მშობლების სწორი მასივი.

მანქანასთან ინტერაქციისთვის, თქვენ შეგიძლიათ გამოიყენოთ შემდეგი ორი პროცედურა.

პირველი პროცედურა:

```
bool insert(int U, int X)
```

- U : ფოთლის ინდექსი, სადაც ბურთულა უნდა ჩაისვას. პირობა $0 \leq U < M$ უნდა იყოს დაკმაყოფილებული.

- X : ბურთულის მთელი ღირებულება. პირობა $0 \leq X \leq 1000$ უნდა იყოს დაკმაყოფილებული.
- ეს პროცედურა აბრუნებს `true`-ს, თუ ბურთულა წარმატებით ჩაისვა, ან `false`-ს, თუ ფოთოლი U უკვე დაკავებული იყო.
- ეს პროცედურა შეიძლება გამოიძახებული იქნას მაქსიმუმ 500 000-ჯერ.

მეორე პროცედურა:

```
std::vector<int> collect()
```

- აგროვებს ყველა ჩასმული ბურთულის მნიშვნელობებს, ცლის მანქანას და აბრუნებს შეგროვებულ მასივს S .

თუ თქვენი პროგრამის შესრულების ნებისმიერ ეტაპზე C -ის მნიშვნელობა გადააჭარბებს 1000-ს, თქვენი ამოხსნა მიიღებს შეფასებას `Output isn't correct: Too many resources used`.

მანქანის სტრუქტურა ფიქსირებულია `find_structure`-ის გამოძახებამდე.

შეზღუდვები

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

ქვეამოცანები

ქვეამოცანა	ქულა	დამატებითი შეზღუდვები
1	5	სათავეს ზუსტად M ცალი შვილი აქვს.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	დამატებითი შეზღუდვები არ არის.

მე-3 და მე-4 ქვეამოცანების შემთხვევაში, თქვენი ქულა დამოკიდებულია C -ის მნიშვნელობაზე შემდეგნაირად.

ქვეამოცანა 3 (25 ქულა)

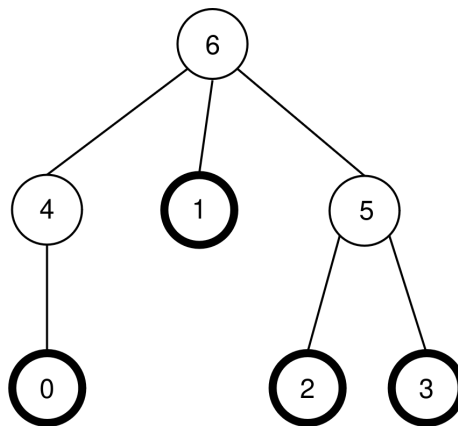
Limits	Score
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

ქვეამოცანა 4 (60 ქულა)

Limits	Score
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

მაგალითი

განვიხილოთ იგივე მანქანა, როგორიც პირობის აღწერაშია. $N = 7$, $M = 4$ და $P = [4, 6, 5, 5, 6, 6]$. მანქანა კიდევ ერთხელ ნაჩვენებია ნახაზზე:



გრეიდერი აკეთებს შემდეგ გამოძახებას:

```
find_structure(4)
```

პროცედურას შეუძლია განახორციელოს გამოძახებების შემდეგი თანმიმდევრობა:

1. **insert(0, 0)**: ფოთოლი 0 თავისუფალია, ამიტომ ბურთულა ღირებულებით 0 თავსდება ფოთოლზე 0. კვანძი $P[0] = 4$ თავისუფალია, ამიტომ ბურთულა გადაადგილდება კვანძზე 4. კვანძი $P[4] = 6$ თავისუფალია, ამიტომ ბურთულა გადაადგილდება კვანძზე 6. რადგან კვანძი 6 სათავეა, გადაადგილება ჩერდება. გამოძახება აბრუნებს `true`.
2. **insert(3, 20)**: ფოთოლი 3 თავისუფალია, ამიტომ 20 მნიშვნელობის ბურთი განთავსდება ფოთოლზე 3. კვანძი $P[3] = 5$ თავისუფალია, ამიტომ ბურთი გადაინაცვლებს კვანძზე 5. რადგან $P[5] = 6$ დაკავებულია, მოძრაობა ჩერდება. გამოძახება აბრუნებს `true`.
3. **insert(1, 10)**: ფოთოლი 1 თავისუფალია, ამიტომ 10 მნიშვნელობის ბურთულა განთავსდება ფოთოლზე 1. რადგან $P[1] = 6$ დაკავებულია, ბურთულა რჩება კვანძში 1. გამოძახება აბრუნებს `true`.

4. `insert(2, 30)`: ფოთოლი 2 თავისუფალია, ამიტომ ბურთი მნიშვნელობით 30 განთავსდება ფოთოლზე 2. რადგან $P[2] = 5$ დაკავებულია, ბურთი რჩება კვანძში 2. გამოძახება აბრუნებს `true`.
5. `insert(1, 25)`: ფოთოლი 1 დაკავებულია, ამიტომ ბურთი არ განთავსდება ფოთოლზე 1. გამოძახება აბრუნებს `false`.
6. `insert(0, 20)`: ფოთოლი 0 თავისუფალია, ამიტომ 20 ღირებულების ბურთულა თავსდება ფოთოლზე 0. კვანძი $P[0] = 4$ თავისუფალია, ამიტომ ბურთულა გადაადგილდება კვანძზე 4. რადგან $P[4] = 6$ დაკავებულია, გადაადგილება წყდება. გამოძახება აბრუნებს `true`.

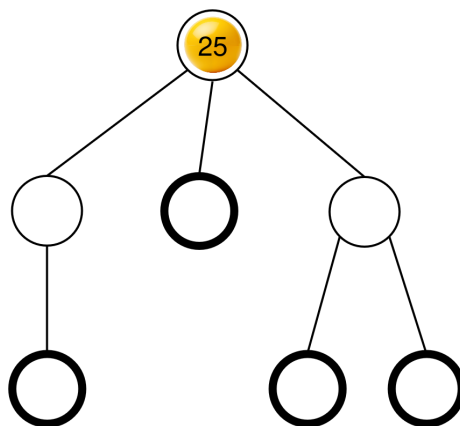
ბურთულების საბოლოო კონფიგურაცია უკვე ნაჩვენები იყო პირობის აღწერაში.

შემდეგ, პროცედურას შეუძლია გამოიძახოს:

```
collect()
```

ამ გამოძახების შესრულება უკვე განმარტებული იყო დავალების აღწერაში, ამიტომ ეს გამოძახება აბრუნებს $S = [0, 10, 20, 30, 20]$. ამის შემდეგ, მანქანა კვლავ ცარიელია.

შემდეგ, პროცედურას შეუძლია გამოიძახოს `insert(2, 25)`. ფოთოლი 2 თავისუფალია, ამიტომ ბურთი ღირებულებით 25 თავსდება ფოთოლზე 0. ეს ბურთი შემდეგ გადაადგილდება კვანძში 5, შემდეგ კი კვანძში 6, სადაც ჩერდება. გამოძახება აბრუნებს `true`. ბურთულების საბოლოო განლაგება ნაჩვენებია შემდეგ სურათზე.



საბოლოოდ, პროცედურას შეუძლია გამოიძახოს `collect()`, რომელიც აბრუნებს $S = [25]$ -ს და აცარიელებს მანქანას.

არსებობს მშობლების ორი სწორი მასივი, რომლებიც `find_structure`-ს შეუძლია დააბრუნოს: $R = P = [4, 6, 5, 5, 6, 6]$ (რომელიც შეესაბამება აღნიშნულს $L = [0, 1, 2, 3, 4, 5, 6]$), და $R = [5, 6, 4, 4, 6, 6]$ (რომელიც შეესაბამება აღნიშნულს $L = [0, 1, 2, 3, 5, 4, 6]$). ამ მაგალითში, $K = 2$ და $B = 30$, ამიტომ $C = 32$.

სანიმუშო გრეიდერი

შეტანის ფორმატი:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

გამოტანის ფორმატი:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

აქ, Q არის მასივის R სიგრძე, რომელიც `find_structure`-მა დააბრუნა.